



rewrite in a report format with objective, intro and details and then conclusion based on the attached as input. provide summaries in table format and flowchart and emphasis paragraph whenever possible to enhance readability.

Objective

To document and explain how login and authorization URLs are constructed and enforced in the libcloud REST + OpenFGA stack, and to describe the exact OpenFGA model, tuples, and bootstrap mechanisms that govern admin, viewer, and proposed auditor access to endpoints such as `http://localhost:8765/v1/compute/nodes`.^[1]

1. Introduction

This report analyzes a local libcloud REST deployment that uses Dex (OIDC) for authentication and OpenFGA for fine-grained authorization.^[1]

The focus is on: (1) the OIDC authorization request URL to Dex, (2) how libcloud REST constructs its internal API URLs, (3) where OpenFGA grants admin/viewer permissions for these APIs, (4) how `openfga_bootstrap.py` is loaded and run, and (5) how to introduce an **auditor** role that can read compute nodes (`GET /v1/compute/nodes`) but cannot provision them (`POST /v1/compute/nodes`).^[1]

2. OIDC Login URL Breakdown

The analyzed Dex authorization request URL is:^[1]

```
http://localhost:5556/dex/auth?client_id=libcloud-  
rest&redirect_uri=http%3A%2F%2F127.0.0.1%3A8766%2Foauth%2Fcallback&response_type=code&scope=o  
penid+email+profile&state=libcloud-dex[1]
```

2.1 Components and Semantics

- Authorization endpoint: `http://localhost:5556/dex/auth` — Dex's OIDC authorization endpoint, running locally on port 5556.^[1]
- `client_id=libcloud-rest` — identifies the OAuth client; Dex must have a registered client `libcloud-rest` with matching redirect URI and secret.^[1]
- `redirect_uri=http://127.0.0.1:8766/oauth/callback` — callback exposed by libcloud REST, where Dex returns code and state after login.^[1]
- `response_type=code` — requests the Authorization Code flow; backend later exchanges code + client secret at `/dex/token`.^[1]

- `scope=openid+email+profile` — OIDC scopes: `openid` for `id_token`, `email` for email claims, `profile` for basic profile info. ^[1]
- `state=libcloud-dex` — anti-CSRF opaque value; stored in session and validated on callback (static here, so weaker than per-request random). ^[1]

2.2 Impact of Skipping Parameters

Libcloud's authentication robustness depends on each parameter. ^[1]

Parameter	Effect if omitted
<code>client_id</code>	Dex rejects request; client not identified; redirect and scopes cannot be validated. ^[1]
<code>redirect_uri</code>	Dex rejects for code flow; browser has nowhere to send <code>code</code> ; login cannot complete. ^[1]
<code>response_type</code>	Dex errors or defaults; without explicit <code>code</code> , token exchange may fail or be non-compliant. ^[1]
<code>scope=openid</code>	No <code>id_token</code> issued; becomes plain OAuth without identity assertions. ^[1]
<code>scope=email/profile</code>	Login works, but tokens/userinfo lack those claims; app only sees a bare subject ID. ^[1]
<code>state</code>	Flow completes but no CSRF protection; enables login-CSRF / confused-deputy attacks. ^[1]

Emphasis: Correctly configured `state` and `openid` scope are critical to preserve both security (CSRF resistance) and identity semantics; removing either degrades the flow from secure OIDC to fragile OAuth with attack surface expansion. ^[1]

3. Libcloud REST URL Construction

Libcloud REST centralizes HTTP call construction in `scripts/common.sh`, with `libcloud_api` responsible for assembling method, path, and headers. ^[1]

3.1 Core Assembler: `libcloud_api`

`scripts/common.sh` defines: ^[1]

```
libcloud_api() {
  local method="$1" path="$2" body="${3:-}"
  local -a extra=()
  if [[ -n "${CONNECTION_PARAM}" ]]; then
    extra+=("-H" "X-Provider-Connection: ${CONNECTION_PARAM}")
  fi
  curl_http "${method}" "${LIBCLOUD_REST_URL}${path}" "${body}" \
    -H "Authorization: Bearer ${ACCESS_TOKEN}" \
    -H "Accept: application/json" \
    "${extra[@]}"
}
```

- `LIBCLOUD_REST_URL` defaults to `http://localhost:8765`, so all paths are prefixed with this base URL. ^[1]
- `Authorization: Bearer ${ACCESS_TOKEN}` injects the Dex-issued JWT. ^[1]
- Optional `X-Provider-Connection` header is added when `CONNECTION_PARAM` is set (e.g., AWS connection tests). ^[1]

3.2 Mapping of Script Steps to URLs

`provision_aws.sh` uses helpers in `common.sh` to construct and invoke specific REST endpoints.^[1]

#	REST URL	Construction point
6	GET /v1/auth/me	<code>provision_aws.sh:54</code> → <code>libcloud_me</code> in <code>common.sh:262-266</code> . ^[1]
7	POST /v1/connections:test	<code>provision_aws.sh:55</code> → <code>libcloud_connection_test "\${connection_json}"</code> . ^[1]
8	GET /v1/compute/locations	<code>provision_aws.sh:58</code> → <code>libcloud_api GET "/v1/compute/locations"</code> . ^[1]
9	GET /v1/compute/sizes	<code>provision_aws.sh:59</code> → <code>libcloud_api GET "/v1/compute/sizes"</code> . ^[1]
10	GET /v1/compute/images?name=%2AUbuntu%2A	<code>provision_aws.sh:60</code> → <code>libcloud_api GET "\${aws_images_path}"</code> . ^[1]
11	GET /v1/compute/nodes	<code>provision_aws.sh:63</code> → <code>libcloud_api GET "/v1/compute/nodes"</code> . ^[1]
12	GET /v1/compute/images?name=%2AUbuntu%2A	<code>provision_aws.sh:79</code> → second <code>aws_images_path</code> call. ^[1]
13	GET /v1/compute/sizes	<code>provision_aws.sh:80</code> → second <code>sizes</code> call. ^[1]
14	GET /v1/compute/subnets	<code>provision_aws.sh:92</code> → <code>libcloud_api GET "/v1/compute/subnets"</code> . ^[1]
15	POST /v1/compute/nodes	<code>provision_aws.sh:112</code> → <code>libcloud_api POST "/v1/compute/nodes" "\${CREATE_BODY}"</code> . ^[1]

Emphasis: All operational compute URLs (locations, sizes, images, nodes, subnets) share the same **policy surface**, as they are uniformly assembled through `libcloud_api` with a Dex JWT and optional provider connection header; this makes OpenFGA the decisive layer for differentiating admin, viewer, and auditor capabilities.^[1]

4. OpenFGA Authorization Model and Role Grants

Authorization logic for these URLs is implemented in `openfga_bootstrap.py` and enforced at runtime by `libcloud REST's policy.py` using OpenFGA checks.^[1]

4.1 Endpoints with and without OpenFGA Checks

Per `authorization.md` and `policy.py`:^[1]

- GET /v1/auth/me (6) and POST /v1/connections:test (7)
 - No OpenFGA checks; they rely solely on JWT and scopes.^[1]
- GET compute routes (8–14: locations, sizes, images, nodes, subnets)
 - Check `can_read` on backend object `aws_region:{auth_binding}` (default `aws_region:aws`), with fallback to `can_provision`.^[1]
- POST /v1/compute/nodes (15)
 - Check `can_provision` on `aws_region:{auth_binding}`.^[1]

4.2 Tenant Role Relations (Location A)

The `tenant` type defines how base roles map to higher-level capabilities:^[1]

```
"can_provision": {
  "union": {
    "child": [
      {"computedUserset": {"relation": "admin"}},
      {"computedUserset": {"relation": "owner"}},
    ]
  }
},
"can_read": {
  "union": {
    "child": [
      {"computedUserset": {"relation": "viewer"}},
      {"computedUserset": {"relation": "admin"}},
      {"computedUserset": {"relation": "owner"}},
    ]
  }
},
},
```

- Admin and owner obtain both `can_provision` and `can_read` through unions.^[1]
- Viewer obtains only `can_read`, not `can_provision`, giving read-only access.^[1]

4.3 Backend Propagation (Location B — `aws_region`)

The `aws_region` type connects tenant roles to backend actions:^[1]

- Relations `tenant_admin`, `tenant_owner`, `tenant_viewer` are `tupleToUserset` mappings from tenant to respective roles.^[1]
- `can_read` on `aws_region` is a union of `viewer`, `operator`, `tenant_viewer`, `tenant_admin`, `tenant_owner`, plus provider-based `can_use`.^[1]
- `can_provision` on `aws_region` is an intersection of:
 - Union of `operator`, `tenant_admin`, `tenant_owner`, and
 - Provider-based `can_use`.^[1]

Thus, admins and owners who can use the provider gain provisioning rights, whereas viewers only gain read rights.^[1]

4.4 Role Tuples (Location C and D)

The **role assignments** are encoded in `INITIAL_TUPLES`:^[1]

```
{"user": "user:aws-owner", "relation": "owner", "object": "tenant:aws"},
{"user": "user:aws-admin", "relation": "admin", "object": "tenant:aws"},
{"user": "user:aws-viewer", "relation": "viewer", "object": "tenant:aws"},
```

And the backend wiring:^[1]

```
{"user": "tenant:aws", "relation": "parent", "object": "provider:aws"},
{"user": "tenant:aws", "relation": "parent", "object": "libcloud_api:main"},
```

```
{ "user": "provider:aws", "relation": "provider", "object": "aws_region:aws" },
{ "user": "tenant:aws", "relation": "tenant", "object": "aws_region:aws" },
```

- `user:aws-admin + tenant:aws + aws_region:aws + provider:aws` together enable both `can_read` and `can_provision` on `aws_region:aws`.^[1]
- `user:aws-viewer` path enables `can_read` only, so read endpoints succeed while provisioning fails.^[1]

4.5 Per-URL Authorization Summary

#	URL	Libcloud check	Object	Admin grant source	Viewer grant source
6	<code>/v1/auth/me</code>	None (JWT only)	—	Not applicable. ^[1]	Not applicable. ^[1]
7	<code>/v1/connections:test</code>	None (JWT scope)	—	Not applicable. ^[1]	Not applicable. ^[1]
8–11	compute GETs	<code>can_read</code>	<code>aws_region:aws</code>	Tenant admin/owner via Location A + B + tuples. ^[1]	Tenant viewer via Location A + B + tuples. ^[1]
12–14	compute GETs (provision path)	<code>can_read</code>	<code>aws_region:aws</code>	Same as 8–11. ^[1]	Viewer typically exits earlier in script. ^[1]
15	POST <code>/v1/compute/nodes</code>	<code>can_provision</code>	<code>aws_region:aws</code>	Intersection of tenant admin/owner and provider.can_use. ^[1]	Viewer excluded from <code>can_provision</code> ; denied. ^[1]

Emphasis: Admin versus viewer behavior is entirely determined by **OpenFGA relations and tuples**, not by libcloud REST route implementations; the REST API only consults `can_read` and `can_provision` on `aws_region:{binding}`, which makes OpenFGA the single source of truth for capability separation.^[1]

5. Bootstrap: Who Loads and Runs `openfga_bootstrap.py`

`openfga_bootstrap.py` is executed as a one-shot bootstrap job that seeds the OpenFGA store, model, tuples, and validation checks.^[1]

5.1 Canonical Path: Docker Compose

`docker-compose.yml` defines an `openfga-bootstrap` service:^[1]

- Image: `python:3.12-slim`.^[1]
- Depends on `openfga` with `service_healthy` condition.^[1]
- Environment: `FGA_API_URL`, `FGA_PUBLIC_API_URL`, `FGA_STORE_NAME`, `SUPERADMIN_JWT`, etc.^[1]
- Volumes: `./openfga_bootstrap.py` and `./generated`.^[1]
- Command: `["python3", "openfga_bootstrap.py"]`.^[1]
- `restart`: `"no"` (run once).^[1]

`setup.sh` triggers this:^[1]

```
echo "Waiting for OpenFGA bootstrap (superadmin-gated) ..."  
docker compose up --no-deps openfga-bootstrap
```

Before that, `scripts/superadmin_auth.sh` logs in via Dex as LLDAP superadmin and exports `SUPERADMIN_JWT`, which the container uses as a gating credential. ^[1]

5.2 Alternate Entry Points

- `myrun.sh`: runs `python3 openfga_bootstrap.py` directly on host for dev re-seeding (requires `SUPERADMIN_JWT` in environment). ^[1]
- `system_validate.sh`: invokes bootstrap with empty `SUPERADMIN_JWT` to assert it exits with code 3 and a gating message. ^[1]
- Manual: `SUPERADMIN_JWT=... python openfga_bootstrap.py`, documented in `authorization.md`. ^[1]

Emphasis: The Python interpreter is the loader, while `SUPERADMIN_JWT` is the actual gatekeeper; without a valid superadmin JWT obtained from Dex, no bootstrap run can mutate the OpenFGA store, ensuring only privileged flows can alter policy and tuples. ^[1]

6. Auditor Role: Making `GET /v1/compute/nodes` Readable but Non-Provisional

The final question is how to configure `http://localhost:8765/v1/compute/nodes` for an auditing role in OpenFGA: i.e., allow read, deny provision. ^[1]

Libcloud REST enforces `GET /v1/compute/nodes` via `can_read` on the backend object, and `POST /v1/compute/nodes` via `can_provision`, so auditor must satisfy the former but not the latter. ^[1]

6.1 Extend Tenant Model with auditor

In `openfga_bootstrap.py`, add an auditor relation to the tenant type and include it in `member` and `can_read`, while keeping `can_provision` untouched: ^[1]

```
"auditor": {"this": {}}, # NEW  
"member": {  
  "union": {  
    "child": [  
      {"this": {}},  
      {"computedUserset": {"relation": "owner"}},  
      {"computedUserset": {"relation": "admin"}},  
      {"computedUserset": {"relation": "viewer"}},  
      {"computedUserset": {"relation": "auditor"}}, # NEW  
    ]  
  }  
},  
"can_read": {  
  "union": {  
    "child": [  
      {"computedUserset": {"relation": "viewer"}},  
      {"computedUserset": {"relation": "auditor"}}, # NEW  
      {"computedUserset": {"relation": "admin"}},  
      {"computedUserset": {"relation": "owner"}},  
    ]  
  }  
},  
},
```

6.2 Propagate Auditor to `aws_region`

Add `tenant_auditor` and include it in `aws_region.can_read`:^[1]

```
"tenant_auditor": { # NEW
  "tupleToUserSet": {
    "tupleSet": {"relation": "tenant"},
    "computedUserSet": {"relation": "auditor"},
  }
},
"can_read": {
  "union": {
    "child": [
      {"computedUserSet": {"relation": "viewer"}},
      {"computedUserSet": {"relation": "operator"}},
      {"computedUserSet": {"relation": "tenant_viewer"}},
      {"computedUserSet": {"relation": "tenant_auditor"}}, # NEW
      {"computedUserSet": {"relation": "tenant_admin"}},
      {"computedUserSet": {"relation": "tenant_owner"}},
      {
        "tupleToUserSet": {
          "tupleSet": {"relation": "provider"},
          "computedUserSet": {"relation": "can_use"},
        }
      },
    ],
  }
},
}
```

Leave `can_provision` unchanged so auditor cannot satisfy provisioning intersection for `aws_region:aws`.^[1]

6.3 Seed Auditor Tuples

Add an auditor principal to `tenant:aws`:^[1]

```
{"user": "user:aws-auditor", "relation": "auditor", "object": "tenant:aws"}, # NEW
```

Existing tuples already connect `tenant:aws` to `aws_region:aws` via `tenant` and to `provider:aws` via `parent`, so the propagation path for auditor is automatically available.^[1]

6.4 Validation Checks

In `VALIDATION_CHECKS`, add assertions to confirm auditor behavior:^[1]

```
("user:aws-auditor", "can_connect", "libcloud_api:main", True),
("user:aws-auditor", "can_use", "provider:aws", True),
("user:aws-auditor", "can_read", "aws_region:aws", True),
("user:aws-auditor", "can_provision", "aws_region:aws", False),
```

6.5 Identity and JWT Layer

OpenFGA is only consulted after authentication and scope checks. [\[1\]](#)

- Create LLDAP user `aws-auditor` via `ensure_user` in `setup.sh` with tenant and role `auditor`.^[1]
- In `identity.py` (`PRINCIPAL_SCOPES`), assign `aws-auditor` `["compute:read"]`, mirroring viewer.^[1]
- In `data/principal_map.json`, map `aws-auditor@libcloud.local` to principal slug `aws-auditor`.^[1]

6.6 Client Script Behavior

provision_aws.sh short-circuits readers/viewers before mutating calls, but does not recognize auditor:
[1]

```
if [[ "${LIBCLOUD_USER}" == "reader" || "${LIBCLOUD_USER}" == "cloud-readonly" || "${LIBCLOUD_USER}" == "cloud-admin" ]]; then
    step "7" "Reader user - skipping mutating AWS provisioning calls"
    exit 0
fi
```

- `LIBCLOUD_USER=aws-auditor` bypasses this reader/viewer guard and can execute `read catalog/node` calls (steps 5–6) but will still be blocked from actual provisioning by OpenFGA's `can_provision` check. ^[1]
- Leaving `PROVISION` unset (dry-run mode) ensures the script does not attempt `POST /v1/compute/nodes`; even if forced, OpenFGA denies auditor provisioning. ^[1]

Emphasis: The **auditor** role is deliberately constructed as a **read-only superset** of viewer semantics at the backend layer, while remaining a **strict subset** of admin/owner with regard to provisioning. This separation is enforced both by OpenFGA's model (no auditor in `can_provision`) and by JWT scope and client-side script guards. ^[1]

7. Summary Tables

7.1 URL, Construction, and Authorization Summary

#	REST URL	Construction site	Auth layer used	Role access (admin/viewer/auditor)
6	GET /v1/auth/me	provision_aws.sh:54 → libcloud_me. [1]	JWT only	Any valid JWT principal. [1]
7	POST /v1/connections:test	provision_aws.sh:55 → connection test. [1]	JWT + scope (compute:read). [1]	Any principal with scope. [1]
8	GET /v1/compute/locations	provision_aws.sh:58 via libcloud_api. [1]	OpenFGA can_read. [1]	admin, owner, viewer, auditor. [1]
9	GET /v1/compute/sizes	provision_aws.sh:59 via libcloud_api. [1]	OpenFGA can_read. [1]	admin, owner, viewer, auditor. [1]
10	GET /v1/compute/images?...Ubuntu...	provision_aws.sh:60 via aws_images_path. [1]	OpenFGA can_read. [1]	admin, owner, viewer, auditor. [1]
11	GET /v1/compute/nodes	provision_aws.sh:63 via libcloud_api. [1]	OpenFGA can_read. [1]	admin, owner, viewer, auditor. [1]

#	REST URL	Construction site	Auth layer used	Role access (admin/viewer/auditor)
12-14	GET images/sizes/subnets	provision_aws.sh:79-92. ^[1]	OpenFGA can_read. ^[1]	admin, owner, auditor (viewer exits earlier). ^[1]
15	POST /v1/compute/nodes	provision_aws.sh:112. ^[1]	OpenFGA can_provision. ^[1]	admin, owner only; viewer/auditor denied. ^[1]

7.2 Role Capability Matrix

Role	Tenant relations	Backend read (can_read)	Backend provision (can_provision)	Example endpoints allowed
Owner	owner, member	Yes via unions. ^[1]	Yes via unions. ^[1]	All read + POST /v1/compute/nodes. ^[1]
Admin	admin, member	Yes via unions. ^[1]	Yes via unions. ^[1]	All read + POST /v1/compute/nodes. ^[1]
Viewer	viewer, member	Yes via unions. ^[1]	No (not in can_provision). ^[1]	Read only; cannot provision nodes. ^[1]
Auditor	auditor, member	Yes (after model change). ^[1]	No (excluded from can_provision). ^[1]	Read nodes, catalogs; cannot create nodes. ^[1]

8. Flowchart: Auditor Access to GET /v1/compute/nodes

Below is a textual flowchart describing the request path for an auditor accessing GET /v1/compute/nodes.^[1]

1. Browser calls Dex authorization URL (/dex/auth?...) and obtains JWT for aws-auditor after successful login.^[1]
2. provision_aws.sh invokes libcloud_api GET "/v1/compute/nodes" with Authorization: Bearer \${ACCESS_TOKEN} and base URL http://localhost:8765.^[1]
3. Libcloud REST receives the request and validates JWT, including compute:read scope for aws-auditor.^[1]
4. policy.py determines backend object aws_region:{auth_binding} (e.g., aws_region:aws) based on environment LIBCLOUD_AWS_AUTH_BINDING.^[1]
5. policy.py issues OpenFGA check: can_read(user:aws-auditor, aws_region:aws).^[1]
6. OpenFGA evaluates:
 - Follows tenant relation from aws_region:aws to tenant:aws.^[1]
 - Resolves auditor role for user:aws-auditor on tenant:aws.^[1]
 - Applies tenant_auditor and can_read unions on aws_region:aws.^[1]
7. OpenFGA returns allow for can_read; libcloud REST responds with node list.^[1]
8. If the same principal attempts POST /v1/compute/nodes, policy.py checks can_provision, which fails because auditor is absent from provisioning unions; request is denied with 403.^[1]

9. Conclusion

The attached configuration demonstrates a layered security architecture where Dex provides OIDC authentication and JWTs, libcloud REST centralizes API URL construction and initial scope checks, and OpenFGA encodes role-based policies for tenants and cloud backends.^[1]

Admin, owner, and viewer behaviors for compute endpoints are driven by OpenFGA relations (`can_read`, `can_provision`) and seeded tuples connecting principals (e.g., `user:aws-admin`, `user:aws-viewer`) to `tenant:aws` and `aws_region:aws`, with provisioning restricted to admin/owner via an intersection that includes provider usage.^[1]

The `openfga_bootstrap.py` script, invoked via Docker Compose, ad-hoc scripts, or manual runs under a superadmin JWT, is the single point where the authorization model, tuples, and validation checks are defined and loaded into OpenFGA, making it the authoritative source for policy.^[1]

By extending the tenant and backend models with an `auditor` relation and propagating it into `aws_region.can_read` while excluding it from `can_provision`, adding corresponding tuples, identity, and scope mappings, the system supports a robust auditing role that can list compute nodes but cannot create them, aligning operational needs with least-privilege principles.^[1]

Is this level of detail and format aligned with how you want to document other flows (e.g., Nutanix or multi-tenant variants), or should I tune the report structure for a more concise “runbook” style?

✱✱

1. full_login_url.txt